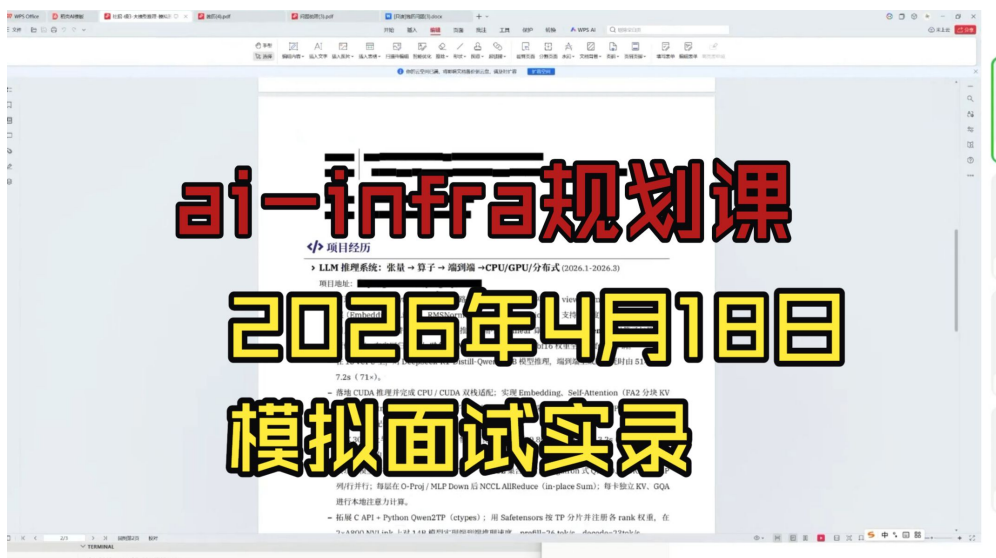


课程笔记

AI-Infra 规划课和模拟面—vLLM 大模型推理优化岗我会怎么面？

五道口纳什 & Codex

2026 年 7 月 18 日



视频作者/频道: 五道口纳什

发布日期: 2026 年 4 月 18 日

视频时长: 45 分 56 秒

视频链接: <https://www.bilibili.com/video/BV1UndWBHE8G/>

目录

1 AI-Infra 行业格局与 vLLM 推理优化岗定位	3
1.1 AI-Infra 是什么	3
1.2 推理优化岗的市场格局	3
1.3 推理框架生态速览	3
1.4 本章小结	4
2 vLLM 推理优化岗面试策略	4
2.1 讲者的面试方法论	4
2.2 PR 驱动面试的特殊路径	4
2.3 本章小结	5
3 项目经验与简历策略	5
3.1 玩具项目的致命伤	5
3.2 社区 PR 的含金量	5
3.3 项目包装的边界	6
3.4 芯片公司实习的「锁死效应」	6
3.5 本章小结	6
4 手撕与八股准备	6
4.1 手撕的三类题型	6
4.2 八股的定位	7
4.3 本章小结	7
5 学习路线与岗位选择	7
5.1 推荐的入门路线	7
5.2 CUDA 学到什么程度	8
5.3 Triton 要不要学	8
5.4 算子开发 vs 推理优化：选哪个	8
5.5 PD 分离与业务场景	9
5.6 本章小结	9
6 三位面试者案例全景	9
6.1 案例一：社招转行者（6 个月求职中）	9
6.2 案例二：研一学生（PD 分离研究方向）	10
6.3 案例三：研二学生（暑期实习求职中）	11
6.4 本章小结	12
7 总结与延伸	12
7.1 讲者的结尾讨论	12
7.2 核心主张与关键机制	12
7.3 实践含义	13
7.4 开放问题	13
7.5 拓展阅读	14

7.6 本章小结	14
--------------------	----

1 AI-Infra 行业格局与 vLLM 推理优化岗定位

1.1 AI-Infra 是什么

AI-Infra (AI 基础设施) 是连接模型算法与硬件的中间层, 核心任务是在给定硬件上高效地训练和部署 AI 模型。随着大模型时代的到来, AI-Infra 已经从传统的「运维支持」角色演变为决定模型落地效率的关键环节。

AI-Infra 内部可细分为以下方向:

- **训练框架**: 分布式训练、混合精度、ZeRO 优化、Pipeline 并行。代表: Megatron-LM、DeepSpeed、PyTorch FSDP
- **推理框架/推理优化**: 模型部署、KV Cache 管理、调度优化、量化部署。代表: vLLM、SGLang、TensorRT-LLM
- **算子开发**: CUDA Kernel 编写与优化、Attention 算子、GEMM 调优。代表: FlashAttention、cutlass
- **模型量化与轻量化**: GPTQ、AWQ、SmoothQuant、FP8/INT4 量化方案
- **芯片 SDK/Compiler**: 将框架模型编译到专用硬件。代表: TensorRT、MLIR

应用层: ChatBot / CodeGen / 文生图 / Agent
模型层: LLaMA / Owen / DeenSeek / Stable Diffusion
AI-Infra 层: 训练框架 · 推理引擎 · 算子库 · 量化工具 · 芯片 SDK
硬件层: NVIDIA GPU / 昇腾 NPU / AMD MI300X / 专用推理卡

图 1: AI-Infra 在技术栈中的定位

1.2 推理优化岗的市场格局

从岗位供需看, 推理优化是当前 AI-Infra 方向中需求量最大的细分岗位。视频中提到上海一个城市就有约 100 个推理相关招聘岗位, 且多数面向中小企业。

推理优化 vs 算子开发: 岗位对比

- **推理优化 (推荐)**: 大厂和互联网公司需求大, 涉及全局优化思维 (调度、KV Cache、PD 分离), 发展路径宽
- **算子开发 (谨慎选择)**: 以国产芯片公司为主, 需求集中但岗位总量偏少, 薪资和天花板偏低。芯片公司对 PD 分离等业务场景优化无需求, 它们提供的是 SDK 而非推理服务

1.3 推理框架生态速览

当前主流推理框架的技术定位差异:

1. **vLLM**: 社区最活跃的通用推理框架, PagedAttention + 连续批处理, 生态完善
2. **SGLang**: 结构化生成见长, RadixAttention + 编程式前端, 学术影响力大

- 3. **TensorRT-LLM**: NVIDIA 官方方案，性能极致但灵活性受限
- 4. **llama.cpp**: CPU/边缘推理标杆，量化方案丰富（GGUF）

视频中讲者明确建议以 vLLM 作为学习入口，理由是社区活跃度高、岗位匹配面广、PD 分离等高级特性与业务场景强耦合。

1.4 本章小结

AI-Infra 的核心是「用系统手段解决模型落地效率」。推理优化岗目前需求最大；vLLM 是推荐的学习入口框架；算子开发需谨慎——它更多是芯片公司的需求，天花板和岗位总数都不如推理优化。

2 vLLM 推理优化岗面试策略

2.1 讲者的面试方法论

视频中讲者系统阐述了他面试候选人的完整策略，分为三个递进层次：

- 1. **八股摸底（定位熟悉模块）**：先通过基础问答了解候选人对 vLLM 哪个模块最熟悉。这不是考核的全部，而是「定位入口」
- 2. **模块深入（验证真实深度）**：针对最熟悉模块深入追问实现原理、设计决策、边界条件。如果候选人有社区 PR，则聚焦 PR 的工作展开——从发现问题到实现和优化的完整链路
- 3. **发散性优化（考察系统思维）**：给一个业务场景（如「推理接口速度慢」）或开放式问题（如「如何优化长文本推理」），考察端到端的定位和优化能力

面试核心逻辑

面试官不期望你每个模块都精通，但期望你在一个模块上有深入理解，并能展现系统性的性能定位能力。八股是入口而非终点；PR 是证明真实深度的最强信号。

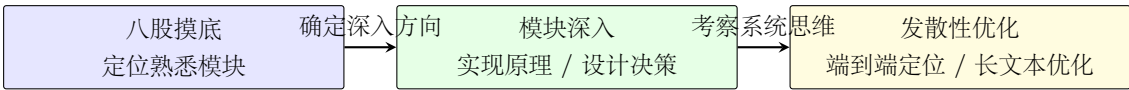


图 2: vLLM 推理优化岗面试三阶段

2.2 PR 驱动面试的特殊路径

对于社招转行或缺乏真实业务经验的同学，给目标框架提交社区 PR 是一条「破局路径」。讲者分享了真实案例：一位从大模型部署应用转行的候选人，因为给 vLLM 提交过两个 PR 而获得面试机会并最终通过。

PR 面试的问题链路：

- 你是怎么发现这个问题的？（问题定位能力）
- 你的实现和优化思路是什么？（方案设计能力）

- 提交 PR 的过程是怎样的? (工程协作能力)
- 有没有被 reviewer 提过修改意见? 你怎么处理的? (迭代能力)

讲者强调, PR 的含金量不在于代码行数多, 而在于**能讲清楚「为什么做、怎么做、如何验证」**的完整 story。

PR 不是万能通行证

纯八股背得好但 PR 一问三不知 → 面试官会质疑 PR 真实性。PR 的价值在于它迫使你真正理解代码, 而不是背面试题。

2.3 本章小结

vLLM 面试的核心是三阶段递进: 模块定位 → 深入验证 → 发散考察。PR 是证明能力的最强信号, 但前提是你能把完整的问题-实现-优化链路讲清楚。发散性优化问题(如端到端性能定位、长文本推理优化)是区分候选人的关键。

3 项目经验与简历策略

3.1 玩具项目的致命伤

视频中三位面试者均面临「玩具项目」困境。讲者指出, 有经验的面试官能一眼识别学习性质的项目与真实业务项目:

1. **缺少业务背景**: 真实项目的优化一定有明确的业务诉求——「推理接口慢、用户投诉」或「某个模型上线后 P99 latency 不达标」。学习项目往往是「为了学 vLLM 而做 vLLM」
2. **优化方向雷同**: P4S CUDA Graph、张量并行 + NCCL 通信——这些是课程标配, 面试官每周能见到好几个写着同样关键词的简历
3. **审美疲劳**: 讲者提到 NanoVM 已经成为高频项目, 「随便点开一个都是自己实现了一个推理框架」

项目真实感

社招对项目经验的要求远高于校招。校招的容忍度稍高, 但如果校招二面被问到「这是实验室项目还是自己做的」就尬住, 说明项目包装的边界已经撞到了。

3.2 社区 PR 的含金量

讲者多次强调, 在缺乏真实业务经验的情况下, 给 vLLM 社区提 PR 是最高含金量的项目加分项。原因:

- **真实性无法造假**: PR 有完整的 review 记录、讨论历史、合入时间线
- **展示完整工程能力**: 发现问题 → 设计方案 → 实现 → 测试 → review 迭代 → 合入
- **代码质量受社区检验**: 能被 vLLM maintainer 接受的代码, 本身就是能力背书

选择 PR 方向的建议: 看 vLLM 的哪些特性可以与你已有的学习项目结合, 在能力范围内选一个能实现的特性去做。

3.3 项目包装的边界

第三位面试官问「项目该怎么包装」，讲者的回答很明确：

- 把 NanoVM 改名叫「自己实现的大模型推理框架」——没用。面试官关心的永远是你做了什么、怎么做的、优化效果是什么
- 把没做过的模块写进简历——更危险。写的越多，准备面越广，被问住的风险越大
- 关键是在现有项目中加入自己的优化：比如 vLLM 有某个特性但 NanoVM 没实现，你去支持一下；或者你支持了一个 NanoVM 不支持的模型

简历膨胀的反噬效应

简历写「一整套大模型推理框架」意味着你需要准备调度器、KV Cache 管理、Continuous Batching、模型支持、量化方案……每个点都可能被深入问。宁可聚焦一个模块做深，也不要把自己摊薄。

3.4 芯片公司实习的「锁死效应」

讲者提到一个值得警惕的职业路径现象：如果在芯片公司实习/工作过，后续找的还是一些芯片企业的实习，会被「一直卡在这里，很难跳去互联网厂商」。

原因：

- 芯片公司的工作内容是 SDK 开发/算子适配，与互联网企业的推理优化岗（端到端性能定位、调度系统优化）的技能栈有差异
- 招聘方的惯性思维——「做过芯片公司的，可能不适应互联网的节奏和问题类型」

但这并不意味着芯片公司绝对不能去——**关键是要问清楚工作内容**。如果是开发 TensorFlow 插件这种纯芯片适配工作，就不值得；如果是做推理框架本身的优化，那就和互联网岗位相通。

3.5 本章小结

项目经验的核心不是「项目名称是什么」，而是「你做了什么优化、怎么做的、效果如何」。社区 PR 是目前最强的项目证明。小心简历膨胀——写的越多，被问住的风险越大。芯片公司的选择需谨慎，关键是岗位工作内容而非公司招牌。

4 手撕与八股准备

4.1 手撕的三类题型

视频中明确区分了三种手撕题：

1. **LeetCode 高频题**：算法和数据结构基础，面试标配，必须熟练
2. **CUDA 算子手撕**：Reduce、MatMul (含 shared memory / bank conflict 优化版本)、LayerNorm 等常见算子。视频中第一位面试者的 MatMul 优化手撕被讲者建议作为重点准备
3. **MHA (Multi-Head Attention) 手撕**：从朴素的 PyTorch 版本到 CUDA Kernel 的完整实现路径

CUDA 算子手撕的具体优化点（视频中提及的优化方向）：

- Shared Memory 的使用（减少 global memory 访问）
- Bank Conflict 优化
- Warp-level 原语
- Tensor Core 利用（wmma / mma）
- FP16/BF16 精度处理

讲者推荐了一篇详细记录 MatMul 从基础 Naive 版本到高性能版本优化思路的文章，建议候选人「把它看熟练，面试时可以把里面的优化策略拿出来跟面试官交流」。

CUDA 算子手撕的准备策略

不是每个算子都需要精通手撕，但一个经典算子（如 MatMul）的完整优化链路必须能讲清楚。从 Naive → Shared Memory → Bank Conflict → Tensor Core，每一步的加速原理和瓶颈分析要烂熟于心。

4.2 八股的定位

讲者澄清了一个常见误解：很多人以为框架面试就是背八股。实际上：

- 八股只是第一个环节——用来定位你熟悉 vLLM 的哪个模块
- 定位之后，面试官会往深里问实现细节、设计权衡、边界情况
- 纯八股熟练但深入问题答不上来 → 面评会很差

所以八股是必要的起点，但不是终点。背诵 vLLM 的模块划分、调度流程、KV Cache 管理机制只是让你能坐到桌边——坐到桌边之后，要看你能不能真正聊出东西。

4.3 本章小结

手撕准备三块：LeetCode 高频题（保持手感）、CUDA 算子（至少精通一个算子的完整优化链路）、MHA 实现。八股是面试入口但非全部，核心是在熟悉模块上能深入阐述原理和设计决策。

5 学习路线与岗位选择

5.1 推荐的入门路线

讲者基于三个面试者的不同背景，给出了统一的学习路线建议：

1. **CUDA 编程基础**：理解 GPU 架构（SM、Warp、Shared Memory、Global Memory）、掌握 CUDA C++ 基础、能手撕常见算子
2. **vLLM 框架系统学习**：PagedAttention 原理、Scheduler 调度机制、Continuous Batching、KV Cache 管理、模型加载流程
3. **选择一个深挖方向**：根据你的研究/项目背景，在 vLLM 的某个模块上达到面试深入水准

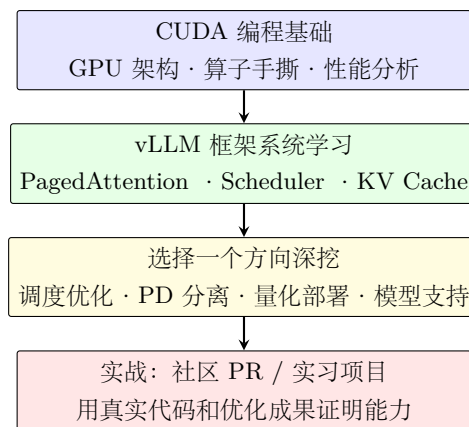


图 3: AI-Infra 推理优化方向学习路线

5.2 CUDA 学到什么程度

讲者对 CUDA 学习深度的定位很务实：

- 目标是「面试时能把常见算子撕出来」，而不是「成为 CUDA 优化专家」
- 但如果面试的是算子开发岗，CUDA 的深度要求会更高
- 推荐方向：**推理优化岗**不需要 CUDA 达到算子专家水平，但必须理解 GPU 执行模型和基本优化方法

对于 PD 分离方向的同学，讲者特别指出 PD 分离必然和 vLLM/SGLang 联合使用，对框架的理解比 CUDA 深度更重要。

5.3 Triton 要不要学

第二位面试者问到 Triton，讲者的回答很明确：

- 三四个内找实习：不用学 Triton。时间紧迫，先把 vLLM 框架熟悉起来
- 有充裕时间：Triton 降低 CUDA 编程门槛，是值得掌握的技能
- 当前就业市场上直接要求 Triton 的岗位并不多

5.4 算子开发 vs 推理优化：选哪个

视频中最具决策价值的对比分析：

维度	算子开发	推理优化
主要雇主	国产芯片公司	互联网大厂 + 中小厂
岗位数量	较少	较多
薪资水平	偏低	偏高
工作内容	CUDA Kernel 编写、适配芯片	端到端性能优化、调度系统
业务关联	弱（不接触推理服务）	强（直接服务业务）
职业跳槽	容易锁定芯片行业	互联网内流转灵活

表 1: 算子开发 vs 推理优化岗位对比

讲者的核心建议：**不是因为硬件限制才去做算子开发，而是因为你对算子本身感兴趣**。如果对算子开发「还好吧，也不是特别感兴趣」，就不要因为缺 GPU 而改方向——缺卡是暂时的，方向选择的影响是长期的。

5.5 PD 分离与业务场景

第二位面试者正在做 PD 分离相关研究（网络感知的调度优化），讲者给出了关键洞察：

- PD 分离本身是一个有真实业务价值的优化方向——互联网企业部署推理服务时，Prefill 和 Decode 阶段的资源需求差异巨大
- 但「网络会议投递的 PD 分离论文」不等于「面试官眼中的 PD 分离经验」——关键在于你**是否理解 vLLM 中 PD 分离的实现和调度细节**
- 芯片公司不需要 PD 分离——因为芯片公司不部署推理服务，它们出售的是 SDK

5.6 本章小结

学习路线：CUDA → vLLM → 深挖一个方向 → 实战证明。CUDA 学到能过面试手撕即可，不必成为专家。Triton 短期内不必要。算子开发谨慎选择，推理优化岗位更优。PD 分离方向有价值但必须和 vLLM 框架深度结合。

6 三位面试者案例全景

本视频的独特价值在于同时呈现了三个不同背景、不同阶段的面试者，形成了一幅完整的 AI-Infra 入行图景。

6.1 案例一：社招转行者（6 个月求职中）

背景：已离职状态，之前有工作经验但非 AI-Infra 方向。每周 3-4 个面试机会，主要面中小企业。

核心问题：

- 项目缺乏真实业务背景，面试官一眼能看出是学习项目
- 手撕准备不足——「感觉还不如群里研二实习生的手撕好」
- 虽然简历有 CUDA 优化相关内容（cutlass、算子优化），但面试时答得不够扎实

讲者诊断：

- 简历本身写得不错，能拿到面试机会说明简历过关
- 核心差距在**项目真实感和手撕熟练度**
- 建议往 vLLM 社区提 PR——PR 是对项目真实性问题的最强回应
- 近期目标是进中小公司，积累真实经验后再考虑跳槽大厂

关键启示：社招的大厂门槛远高于校招，但不是没有路。先入行（中小公司）→ 积累真实项目经验 → 再跳大厂，是一条可行路径。

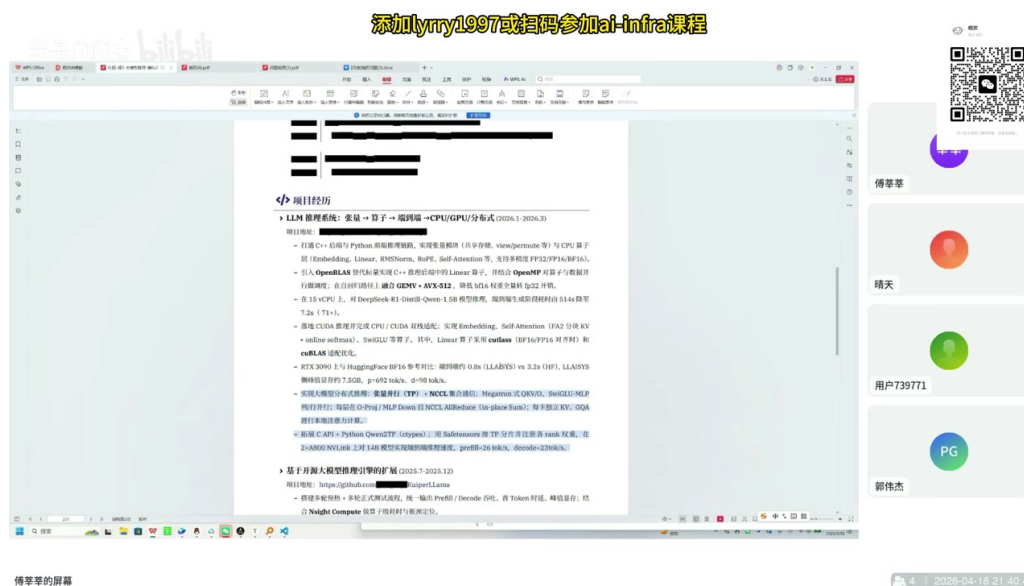


图 4：第一位面试者（社招转行者）的模拟面场景¹

6.2 案例二：研一学生（PD 分离研究方向）

背景：研一，工作过一年后读研，导师指定 PD 分离方向，实验室资源有限（这学期无卡可用）。

核心问题：

- 论文投网络会议还是系统会议？——怕网络会议不被 AI-Infra 面试官认可
- 无卡如何做框架研究？
- 要不要因为缺卡而转向算子开发 + 模型量化？

讲者诊断：

- **继续投网络会议**——面试官关注的是你做了什么工作、解决了什么问题，不纠结会议类型。降级去投系统会议不划算
- **不要因为硬件限制改方向**——对算子「不是特别感兴趣」就不必勉强。后续实习时用公司的卡
- PD 分离的论文工作完全可以作为项目经验写进简历——「你做的工作、你的优化思路」才是面试官想看到的
- **必须学 vLLM**——PD 分离在生产中必然和 vLLM/SGLang 联合使用，不懂框架会「挺尴尬的」

关键启示：研一就有方向意识是优势。硬件限制是暂时的（实习时有公司的卡），方向选择是长期的。PD 分离的方向和推理优化岗天然对口，不需要因为会议类型焦虑。

¹ 视频画面时间区间：00:08:05-00:08:15。

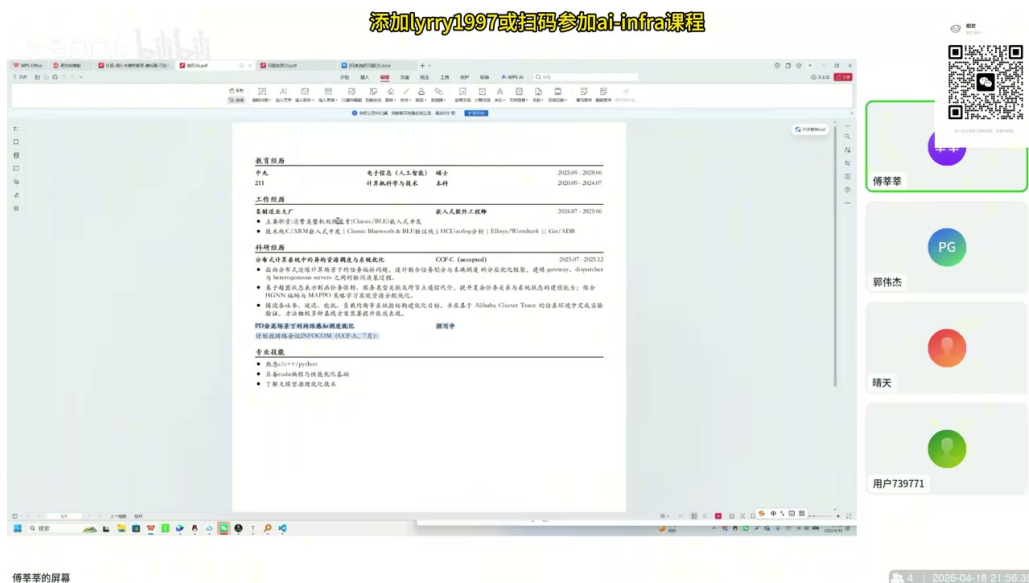


图 5: 第二位面试官（研一 PD 分离方向）的模拟面场景²

6.3 案例三：研二学生（暑期实习求职中）

背景：「二进宫」同学——今年 1 月咨询过一次，现在找暑期实习。一面能过，二面被淘汰。

核心问题：

- 项目是 NanoVM + 简单优化（CUDA Graph 支持），面试官「审美疲劳」
- 二面被问到「这个项目是实习中的还是自己做的」就尬住
- 简历中的 Profiling 工作太常规，看不出差异化
- 在摩尔线程和无问苍穹两个实习 offer 之间犹豫

讲者诊断：

- 关键不是包装——改个项目名字没有用。必须在项目里**加入自己的优化**：比如支持一个 NanoVM 没有的 vLLM 特性，或支持一个 NanoVM 不支持的模型
- 手上已有中小厂 offer 的同学：**先干着**。实习随时能跑路，积累真实经验比等不确定的池子更重要
- 摩尔线程 vs 无问苍穹：讲者倾向无问苍穹。理由是芯片公司的工作可能被锁定在芯片行业，且要问清楚工作内容——如果去摩尔线程是开发 TensorFlow 插件，那就不值得
- 无问苍穹虽然规模小（100+ 人），但「声量不错，算明星初创」，说明创始团队技术背景强

关键启示：简历上写「Profiling」不等于面试官能看到你的工作。你的优化必须**具体、可见、有量化效果**。实习先干着积累经验，不要空等不确定的机会。

²视频画面时间区间：00:24:00-00:24:10。

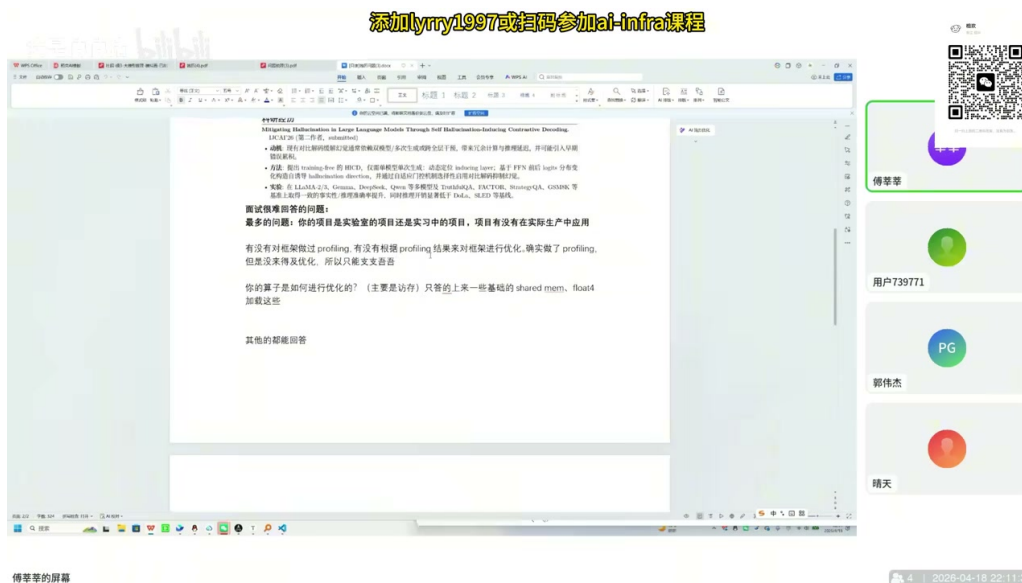


图 6: 第三位面试者（暑期实习求职）的模拟面场景³

6.4 本章小结

三位面试者代表了 AI-Infra 入行的三种典型路径：社招转行（先入行再深耕）、研一规划（方向正确 + 时间充裕）、研二冲刺（差异化优化 + 先干再说）。共同教训是：项目真实感、手撕熟练度、方向选择，三者缺一不可。

7 总结与延伸

7.1 讲者的结尾讨论

视频末尾，讲者在三位面试者的对话之外给出了一些整体性的观察：

- 中小公司对 AI-Infra 人才的需求正在快速增长——「上海大概有 100 个左右的推理招聘岗位」，且这个数字在持续上升
- 模型量化方向「吸收就业」的能力不如推理优化——量化方法趋于成熟（常用就那么几种），除非做量化算法本身的改进（偏算法岗）
- 面试中讲者最看重的是**系统思维**——不是能背多少八股，而是面对一个不熟悉的性能问题，能不能有条理地定位和优化

7.2 核心主张与关键机制

从本视频中蒸馏出的核心命题：

³视频画面时间区间：00:39:00–00:39:10。

AI-Infra 入行三支柱

1. **方向选择决定天花板**：推理优化 > 算子开发；互联网 > 芯片公司（除非你对芯片有特殊兴趣）
2. **项目真实感是硬通货**：社区 PR > 实习项目 > 学习项目。真实感的本质是「能讲清楚为什么做、怎么做、效果如何」
3. **系统思维是终极竞争力**：面试的高阶区分点不在八股背诵，而在于面对一个开放性问题（如「推理慢了怎么定位」）时的分析框架

7.3 实践含义

如果你在准备面试：

- 找一个 vLLM 的 issue，试着提一个 PR——哪怕只是修一个 bug 或加一个小 feature
- CUDA 算子手撕至少精通一个（MatMul 的完整优化链路是最安全的选择）
- 准备一个「端到端性能定位」的思维框架：profiling → 定位瓶颈 → 分析原因 → 设计优化方案 → 验证效果

如果你是研一/研二学生：

- 尽早确定细分方向（推理优化 vs 算子开发 vs 量化），不要因为一时的硬件限制改方向
- 论文工作完全可以包装为项目经验——关键是展示你的分析和优化能力
- PD 分离、调度优化等方向与 vLLM 天然耦合，学框架是必须的

如果你在考虑 offer：

- 问清楚岗位的工作内容——「推理框架优化」可能是做真正的推理引擎优化，也可能只是给芯片写 TensorFlow 插件
- 芯片公司经历可能锁定职业路径，但如果工作内容是推理框架本身（而非芯片适配），同样有价值
- 实习随时能跑路——先干着积累真实经验，不要空等不确定的机会

7.4 开放问题

视频没有覆盖但值得思考的方向：

- **SGLang vs vLLM**：讲者提到了 SGLang 但也将其和 vLLM 并列。如果时间允许，是否应该两个框架都了解？SGLang 的 RadixAttention 和结构化生成在什么场景下优于 vLLM？
- **Attention 架构演进**：MLA（Multi-head Latent Attention，DeepSeek-V2/V3）对 KV Cache 管理的冲击——当 KV Cache 大小从 MB 级骤降到 KB 级，传统的 PagedAttention 是否还需要？
- **PD 分离的边界**：什么场景下 PD 分离是正收益？什么场景下反而是负优化（通信开销 > 并行收益）？这需要在实际业务中积累判断力

- **推理服务化趋势**：推理框架从库到服务的演进（vLLM 的 OpenAI-compatible API Server、SGLang 的 Runtime）对岗位技能要求的影响

7.5 拓展阅读

- vLLM 官方文档: <https://docs.vllm.ai/> ——从 PagedAttention 论文到最新特性的最佳入口
- Efficient Memory Management for Large Language Model Serving with PagedAttention ——vLLM 核心论文
- SGLang: Efficient Execution of Structured Language Model Programs ——SGLang 论文, 了解 RadixAttention 和结构化生成
- CUDA C++ Programming Guide: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> ——CUDA 学习的官方基础

7.6 本章小结

AI-Infra 推理优化方向正处于需求爆发期。入行路径清晰: CUDA 打底 → vLLM 框架 → 深挖一个方向 → 实战证明。不同背景（社招转行/研一规划/研二冲刺）有各自的策略侧重，但共同的要求是：方向选择要慎重、项目要有真实感、系统思维是终极竞争力。先入行再深耕，是多数人最务实的路径。